THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG

1 Giới thiệu về Thiết kế Giao diện Người dùng (UI)

1.1 Giới thiệu

Thiết kế giao diện là quá trình xác định cách hệ thống tương tác với các thực thể bên ngoài (external entities), chẳng hạn như khách hàng, nhà cung cấp, hoặc các hệ thống khác. Nội dung của chương này là thiết kế giao diện người dùng (User Interface), tức là cách người dùng tương tác với hệ thống. Cần lưu ý rằng ngoài giao diện người dùng, còn có giao diện hệ thống (system interfaces), nơi thông tin được trao đổi với các hệ thống khác.

Các thành phần cơ bản của giao diện người dùng:

- Điều hướng (Navigation mechanism): Là cách người dùng đưa ra chỉ dẫn cho hệ thống và cho hệ thống biết phải làm gì (ví dụ: các nút (buttons) và menu)
- Đầu vào (Input mechanism): Là cách hệ thống thu thập thông tin (ví dụ: các biểu mẫu (forms) để thêm khách hàng mới)
- Đầu ra (Output mechanism): Là cách hệ thống cung cấp thông tin cho người dùng (ví dụ: các báo cáo (reports))

Cần lưu ý rằng: *Người dùng chỉ nhận biết hệ thống thông qua UI. Do vậy, thiết kế UI cực kỳ quan trọng!*

1.2 Tính khả dụng

Trong quá trình phát triển hệ thống thông tin, người phân tích hệ thống thường quan tâm đến nhiều yếu tố kỹ thuật khác nhau như dữ liệu cần thu thập, lưu trữ, xử lý; các quy trình nghiệp vụ cần được tự động hóa; hay các quy tắc kiểm soát thông tin. Tuy nhiên, đối với người sử dụng, giao diện người dùng (User Interface – UI) chính là "bộ mặt" của hệ thống, là cách duy nhất mà họ tương tác và cảm nhận về hệ thống đó.

Người dùng thường không nhận thức được sự phức tạp hay tính hiệu quả của các nền tảng kỹ thuật bên dưới; họ chỉ quan tâm đến việc hệ thống có giúp họ hoàn thành công việc một cách dễ dàng và thuận tiện hay không. Do vậy, giao diện người dùng chính là cầu nối giữa người sử dụng và toàn bộ hệ thống thông tin.

Mục tiêu của quá trình thiết kế giao diện là tạo ra một hệ thống có tính khả dụng cao, nghĩa là người dùng có thể thực hiện các tác vụ cần thiết một cách nhanh chóng, chính xác và hiệu quả.

1.2.1 Khái niệm về tính khả dụng

Khái niệm tính khả dụng là trung tâm của thiết kế UI. Theo chuẩn ISO 9241-11, tính khả dụng được hiểu là "mức độ mà người dùng có thể sử dụng một sản phẩm để đạt được mục tiêu cụ thể với hiệu quả, hiệu suất và sự hài lòng trong một ngữ cảnh sử dụng xác định".

Tác giả Alan Dennis và cộng sự (2021) nhấn mạnh rằng tính khả dụng bao gồm hai thành phần có quan hệ chặt chẽ:

- *Dễ sử dụng (Ease of Use):* Hệ thống cho phép người dùng hoàn thành các tác vụ một cách nhanh chóng, thuận tiện và chính xác → Nhấn mạnh đến hiệu quả trong quá trình thao tác.
- *Dễ học (Ease of Learning):* Hệ thống giúp người dùng mới dễ dàng nắm bắt và ghi nhớ các chức năng, quy trình thao tác mà không cần nhiều hướng dẫn → Nhấn mạnh đến tốc độ làm quen và khả năng tiếp cận.

Một hệ thống có tính khả dụng cao cần phải đồng thời đạt được cả hai yếu tố này — vừa dễ học, vừa dễ dùng — để đảm bảo rằng người dùng ở mọi mức độ kinh nghiệm đều có thể thao tác hiệu quả.

1.2.2 Tầm quan trọng của tính khả dụng

Lợi ích của một giao diện có tính khả dụng cao:

- *Giảm nỗ lực của người dùng (Reduce User Effort):* Giao diện được thiết kế tốt cho phép người dùng tập trung vào công việc, thay vì phải suy nghĩ cách thao tác với hệ thống.
- *Tăng tốc độ hoàn thành công việc (Increase Task Efficiency):* Các thao tác được rút gọn và bố trí hợp lý giúp rút ngắn thời gian hoàn thành tác vụ.
- *Giảm sai sót (Reduce Errors):* Giao diện trực quan, rõ ràng giúp người dùng ít mắc lỗi hơn, đồng thời dễ dàng phát hiện và sửa lỗi khi cần.

Ngoài các lợi ích trên, tính khả dụng cao còn giúp:

- Giảm chi phí đào tạo người dùng mới.
- Tăng sự hài lòng và gắn bó của người sử dụng.
- Nâng cao hình ảnh chuyên nghiệp và độ tin cậy của tổ chức.

1.3 Mối quan hệ giữa giao diện người dùng và trải nghiệm người dùng (User Experience – UX)

Giao diện người dùng (UI) tập trung vào hình thức và cấu trúc trình bày của hệ thống: các nút bấm, biểu tượng, menu, biểu mẫu, màu sắc, font chữ, bố cục, ...

Trải nghiệm người dùng (UX) lại mang nghĩa rộng hơn, bao gồm toàn bộ cảm nhận, thái độ và phản ứng của người dùng khi tương tác với hệ thống. UX chịu ảnh hưởng bởi cả UI, nội dung, tốc độ phản hồi, độ tin cậy, và mức độ thỏa mãn nhu cầu của người dùng.

Mối quan hệ giữa hai khái niệm này có thể hình dung như sau:

Khía cạnh	UI (User Interface)	UX (User Experience)
Trọng tâm	Thiết kế hình thức và điều khiển	Cảm nhận, hiệu quả, cảm xúc người dùng
Mục tiêu	Tạo giao diện dễ sử dụng, hấp dẫn	Đảm bảo trải nghiệm tích cực và giá trị
Kết quả	Sản phẩm hiển thị	Trải nghiệm tổng thể của người dùng

Một giao diện tốt là điều kiện cần cho trải nghiệm tốt, nhưng chưa đủ. Người thiết kế hệ thống phải hiểu rằng mục tiêu cuối cùng không chỉ là "giao diện đẹp" mà là "người dùng hài lòng và đạt được mục tiêu của họ dễ dàng".

2 Các nguyên tắc cơ bản trong thiết kế giao diện

Dưới đây là một số nguyên tắc và hướng dẫn cơ bản để góp phần tạo ra một giao diện người dùng có tính khả dụng cao.

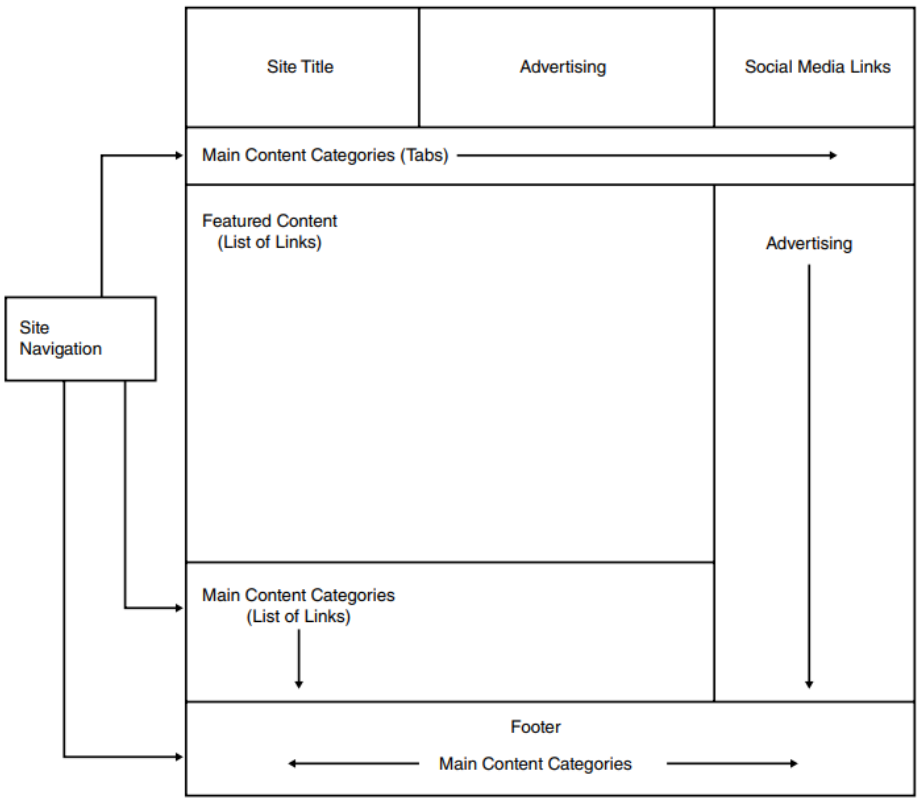
2.1 Bố cục (Layout)

Bố cục đề cập đến việc tổ chức các khu vực của màn hình, biểu mẫu hoặc báo cáo cho các mục đích khác nhau và sử dụng các khu vực đó một cách nhất quán trong toàn bộ giao diện người dùng.

2.1.1 Cấu trúc khu vực màn hình

Hầu hết các phần mềm được thiết kế cho máy tính cá nhân tuân theo cách tiếp cận tiêu chuẩn, chia màn hình thành ba khu vực chính:

- Khu vực trên cùng (Top area): Cung cấp các cơ chế điều hướng (navigation) và lệnh (command).
- Khu vực giữa (Middle area): Khu vực lớn nhất, dành cho việc hiển thị công việc hoặc thông tin đầu vào/đầu ra của người dùng.
- Khu vực dưới cùng (Bottom area): Chứa thông tin trạng thái (status information) về những gì người dùng đang làm



Hình 1 Ví dụ về bố cục một trang web

2.1.2 Luồng thông tin tự nhiên

Các khu vực và thông tin bên trong chúng nên có một luồng tự nhiên để giảm thiểu sự di chuyển của người dùng từ khu vực này sang khu vực tiếp theo.

- Người dùng ở các quốc gia phương Tây thường đọc từ trên xuống dưới, từ trái sang phải. Thông tin liên quan nên được sắp xếp theo thứ tự này (ví dụ: dòng địa chỉ, tiếp theo là thành phố, tiểu bang/tỉnh, và sau đó là mã zip/mã bưu điện).
- Thứ tự có thể dựa trên thứ tự thời gian (chronological order), từ tổng quát đến cụ thể, hoặc từ thông tin được sử dụng thường xuyên nhất đến ít thường xuyên nhất.

2.1.3 Tính nhất quán trong bố cục

Bố cục giao diện cần được duy trì nhất quán về kích thước, hình dạng và vị trí của các khối nội dung trên toàn hệ thống. Điều này giúp người dùng dễ dàng nhận biết và thao tác, giảm thời gian học cách sử dụng giao diện.

Lý tưởng nhất, các thành phần như biểu mẫu nhập liệu (input forms) và báo cáo xuất dữ liệu (output reports) nên được sắp xếp theo cùng một cấu trúc và quy tắc bố trí, để mang lại trải nghiệm trực quan và thống nhất.

2.2 Nhận thức Nội dung (Content Awareness)

Nhận thức nội dung đề cập đến khả năng của giao diện giúp người dùng nhận biết được nội dung thông tin đang được hiển thị và vị trí của họ trong hệ thống.

- Tiêu đề và Tiêu đề Phụ: Tất cả các giao diện nên có tiêu đề rõ ràng để người dùng biết họ đang ở đâu.
- Vị trí trong hệ thống: Giao diện nên bao gồm các yếu tố giúp người dùng nhận ra vị trí hiện tại của họ trong cấu trúc hệ thống. Ví dụ, trong các menu hoặc trang web, có thể sử dụng breadcrumbs để hiển thị đường dẫn điều hướng (ví dụ: Clients >> Flight Requests >> Edit Request).



Clients >> Flight Requests >> Edit Request

- Khối nội dung rõ ràng: Các khối bên trong biểu mẫu và báo cáo nên rõ ràng và được xác định rõ (nên có tiêu đề nếu không gian cho phép) để giảm thiểu sự nhầm lẫn.
- Nhãn (Field Labels):
    - Nhãn dùng để xác định các trường dữ liệu phải ngắn gọn và cụ thể.
    - Cần sử dụng thuật ngữ quen thuộc với người dùng.
    - Cần cung cấp giải thích rõ ràng cho bất kỳ trường nào có khả năng gây mơ hồ (ví dụ: làm rõ định dạng ngày tháng (tháng/ngày/năm so với ngày/tháng/năm) hoặc đơn vị tiền tệ (US so với HK))
- Thông tin báo cáo: Tất cả các biểu mẫu và báo cáo nên có ngày tạo và số phiên bản (version numbers) để người dùng có thể nhận dạng các tài liệu hoặc phần mềm đã lỗi thời.

2.3 Tính thẩm mỹ (Aesthetics)

Tính thẩm mỹ đề cập đến việc giao diện có hấp dẫn và dễ chịu về mặt thị giác hay không.

- Sử dụng khoảng trống (White Space): Việc sử dụng khoảng trống một cách thích hợp là rất quan trọng để tránh tạo ra cảm giác lộn xộn trên giao diện.
- Mật độ (Density): Mật độ thông tin trên màn hình thường nên thấp hơn so với trên báo cáo giấy. Nếu một giao diện quá dày đặc thông tin, nó có thể khiến người dùng cảm thấy quá tải và khó tìm thấy thông tin cần thiết.

- Phông chữ (Font): Nên sử dụng một phông chữ nhất quán cho toàn bộ biểu mẫu hoặc báo cáo và không quá hai kích cỡ phông chữ.
    - Phông chữ Serif (ví dụ: Times Roman) dễ đọc nhất cho các báo cáo được in.
    - Phông chữ Sans Serif (ví dụ: Tahoma hoặc Arial) dễ đọc nhất cho màn hình máy tính và thường được dùng cho tiêu đề trong các báo cáo in.
    - Tránh sử dụng chữ in nghiêng (italics) và gạch chân (underlining) vì chúng làm cho văn bản khó đọc hơn.
    - Tránh sử dụng toàn bộ chữ in hoa (all capital letters), ngoại trừ có thể là tiêu đề, vì chúng làm cho văn bản khó đọc hơn.
- Màu sắc (Color): Màu sắc phải được sử dụng cẩn thận vì nó có thể làm phân tâm hoặc gây nhầm lẫn. Việc sử dụng màu sắc không nhất quán có thể làm giảm tính thẩm mỹ và phá vỡ tính nhất quán của giao diện.

2.4 Mức độ Sử dụng (Usage Level)

Giao diện cần được thiết kế để cân bằng giữa nhu cầu của người dùng thường xuyên và người dùng không thường xuyên.

- Người dùng thường xuyên muốn một hệ thống được thiết kế để dễ sử dụng (rút ngắn các bước, hiệu quả, tốc độ nhanh).
- Người dùng không thường xuyên cần một hệ thống được thiết kế để dễ học (cung cấp hướng dẫn rõ ràng, dễ hiểu)

Một số chiến lược để đáp ứng cả hai nhóm người dùng:

Nhóm Người dùng	Nhu cầu	Chiến lược Thiết kế
Người dùng thường xuyên	Tốc độ, hiệu quả	Cung cấp phím nóng (hot keys), cử chỉ (gestures), hoặc lối tắt (shortcuts) để bỏ qua các menu dài.
Người dùng không thường xuyên	Dễ học, hướng dẫn	Cung cấp các menu (menus) và hướng dẫn rõ ràng (help messages).

2.5 Tính nhất quán (Consistency)

Tính nhất quán là yếu tố quan trọng nhất trong việc làm cho hệ thống trở nên đơn giản để sử dụng, vì nó cho phép người dùng dự đoán điều gì sẽ xảy ra.

- **Tính nhất quán về điều hướng**: Vị trí của các menu, nút lệnh, và quy trình nhập dữ liệu nên nhất quán trên toàn bộ hệ thống.
- **Tính nhất quán về thuật ngữ:** Thuật ngữ được sử dụng trên các menu, nhãn, thông báo và báo cáo phải nhất quán. Ví dụ: Nếu sử dụng thuật ngữ "Client" (Khách hàng), không nên chuyển sang dùng "Customer" (Khách hàng) ở màn hình khác.

- **Tính nhất quán về thiết kế:** Định dạng của các biểu mẫu, báo cáo, và màn hình nên thống nhất.

Tuy nhiên, đôi khi có thể phá vỡ tính nhất quán để thu hút sự chú ý của người dùng, ví dụ như để nhấn mạnh một thông báo lỗi nghiêm trọng hoặc một tính năng mới.

2.6 Giảm thiểu Nỗ lực của Người dùng (Minimize User Effort)

Nguyên tắc này tập trung vào việc giảm thiểu các bước cần thiết để người dùng hoàn thành một tác vụ.

- Quy tắc ba cú nhấp chuột (Three-clicks rule): Người dùng có thể truy cập thông tin hoặc thực hiện chức năng mong muốn chỉ với tối đa ba lần nhấp chuột (hoặc thao tác gõ phím).

2.7 Các vấn đề Đặc biệt của Thiết kế Giao diện Màn hình Cảm Ứng

Màn hình cảm ứng có các thách thức thiết kế riêng biệt.

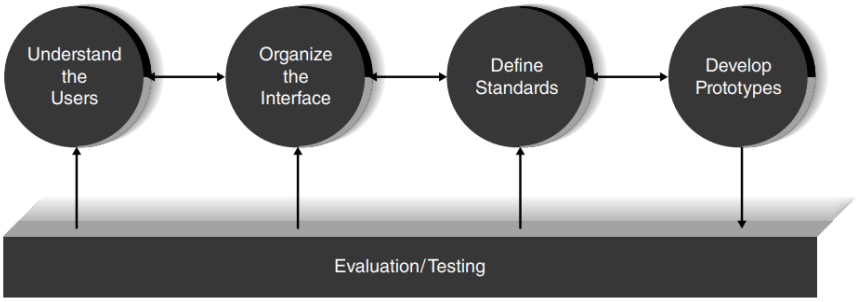
- **Vị trí của các yếu tố điều hướng**: Các yếu tố điều hướng nên được đặt ở cuối màn hình để dễ dàng tiếp cận bằng ngón cái.
- **Ngăn chặn lỗi**: Các nút bấm phải được thiết kế đủ lớn và có đủ khoảng cách để ngăn người dùng nhấp nhầm hoặc nhấp đúp (double-click) ngoài ý muốn.
- **Cử chỉ chuẩn hóa**: Nên sử dụng các tương tác cử chỉ đã được chuẩn hóa (ví dụ: chạm (tap), vuốt (swipe), kéo (drag)) vì người dùng đã quen thuộc với chúng.

3 Quy trình thiết kế UI

Thiết kế giao diện người dùng là một quy trình gồm năm thành phần mang tính lặp lại. Mục tiêu của quy trình này là thiết kế một giao diện đạt mức tính khả dụng cao. Năm thành giai đoạn chính của quy trình thiết kế giao diện người dùng bao gồm:

1) Hiểu người dùng
2) Tổ chức giao diện
3) Xác định tiêu chuẩn
4) Tạo mẫu thiết kế UI
5) Đánh giá/Kiểm thử giao diện

Nguyên tắc quan trọng trong quy trình thiết kế UI: "*kiểm thử sớm; kiểm thử thường xuyên*"



Hình 2 Quy trình thiết kế UI

3.1 Hiểu người dùng

Trong giai đoạn này, điều quan trọng là phải đảm bảo rằng nhóm phát triển hiểu rõ về đối tượng người dùng của hệ thống, đặc biệt nếu người dùng là các đối tượng bên ngoài tổ chức.

- **Personas**: là tổng hợp các đặc điểm chung của một nhóm người dùng cụ thể. Bằng cách xác định personas dựa trên sở thích, hành vi điển hình, mục tiêu và kỳ vọng, người phân tích có thể lập kế hoạch cho một giao diện người dùng làm hài lòng nhóm người dùng đó.
- **Kịch bản Sử dụng**: là một dàn ý các bước mà người dùng thực hiện để hoàn thành một phần công việc của họ. Một kịch bản sử dụng được xây dựng dựa trên các luồng xử lý trong đặc tả use case.

Hãy đọc kĩ ví dụ dưới đây để hiểu rõ hơn về Personas và Kịch bản sử dụng.

Martin: wants to learn details about DrōnTeq's products	
"I'm focused on finding the best commercial drone for my business needs."	
Knowledge:	Understands commercial drones and has specific uses and capabilities in mind for the drone purchase.
Tasks:	Arrives a site to select and configure custom drones. Wants to be able to easily identify drone models and options to meet his intended purposes. Wants to save and compare several configurations in terms of capabilities and prices.
Interests:	Has a business interest in commercial drones. Wants to find the best tool to use to fulfill business needs. Cost/benefit analysis is important.
Characteristics:	Deliberate; thoughtful; expects to be able to develop and compare alternative configurations easily and seamlessly.
Jesse: wants to learn about the uses of commercial drones	
"I'm intrigued by the possibilities of commercial drones and want to learn more."	
Knowledge:	Extremely limited specific knowledge of commercial drone uses and capabilities. Most likely knowledgeable about hobby-level drones. Wants to explore through examples, case studies, video clips, and owner descriptions of commercial drone applications.
Tasks:	Arrives at site and looks for video clips of drones performing commercial operations: photography, agricultural surveys, pipeline inspections, security surveillance, etc. Within each usage category, wants to discover more details through product descriptions, examples, and owner comments.
Interests:	Has a general interest in commercial drone applications and prefers to see an array of ideas and examples.
Characteristics:	Curious, open-minded; creative, exploratory. Wants to get excited by the possibilities offered by commercial-level drones.

Hình 3 Ví dụ về personas

Use Scenario: Focused Visitor	Use Scenario: Exploratory Visitor
User wants to easily develop alternative drone configurations and compare price and performance of the alternatives	User wants to learn about applications of commercial drones through video clips and written material
1. User selects a drone base model 2. User looks at lists of available options for that model 3. User selects options and wants to learn price and capabilities of that configuration 4. User saves, modifies, or discards that configuration and repeats with other options or other drones 5. User has one or more configurations to save and consider for purchase. 6. User may want to contact a sales representative with questions.	1. User views video clips of drones being used in commercial applications 2. User selects an application area and views more clips in that area 3. User reads site marketing material 4. User reads owner descriptions and comments 5. User repeats steps 2-4 for other commercial application areas. 6. User views list of drone models 7. User selects a model and reads site marketing material for that drone.

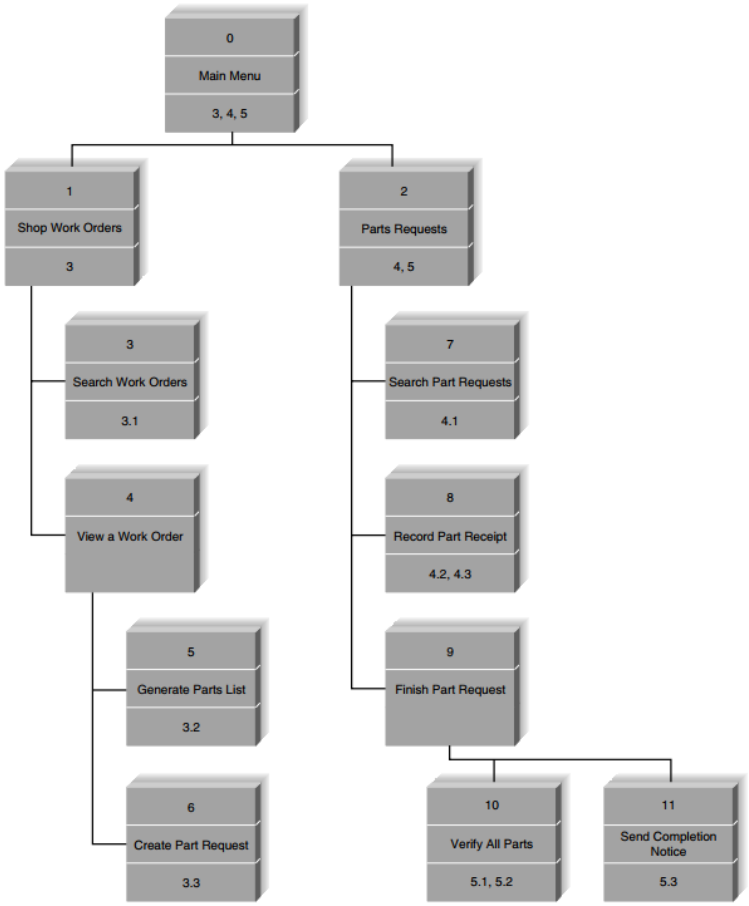
Hình 4 Ví dụ về kịch bản sử dụng

3.2 Tổ chức giao diện

Thiết kế cấu trúc giao diện xác định các thành phần cơ bản của giao diện và cách chúng phối hợp với nhau để cung cấp chức năng cho người dùng.

3.2.1 Sơ đồ Cấu trúc Giao diện (Interface Structure Diagram - ISD)

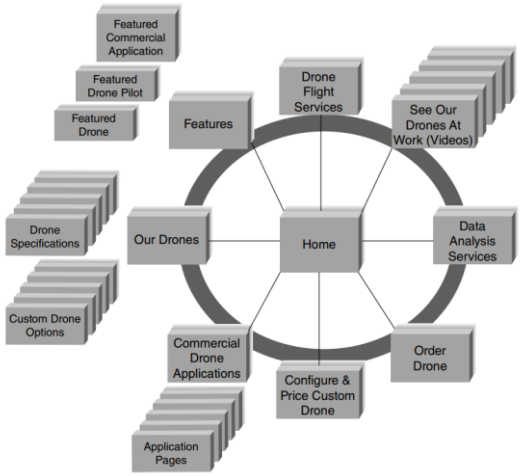
ISD hay còn được gọi là sơ đồ luồng màn hình được sử dụng để hiển thị cách tất cả các màn hình, biểu mẫu và báo cáo được sử dụng bởi hệ thống có liên quan với nhau và cách người dùng di chuyển từ cái này sang cái khác. Sơ đồ này thường có cấu trúc phân cấp (hierarchy) hoặc dạng cây (tree).



Hình 5 Ví dụ về ISD

3.2.2 Bản đồ Trang (Site Map)

Khi phát triển một website, bản đồ trang là một công cụ hữu ích để tổ chức nội dung và giúp xác định hệ thống phân cấp thông tin trên website. BA nên xem xét và đánh giá bản đồ trang cẩn thận, tưởng tượng người dùng (personas) truy cập trang và sử dụng cấu trúc để hoàn thành mục tiêu của họ.



Hình 6 Ví dụ về sitemap

3.3 Xác định Tiêu chuẩn

Các tiêu chuẩn giao diện là các yếu tố thiết kế cơ bản phổ biến trên các màn hình, biểu mẫu và báo cáo riêng lẻ trong hệ thống. Các tiêu chuẩn này đảm bảo rằng các giao diện là nhất quán trong toàn bộ hệ thống.

3.3.1 Ẩn dụ Giao diện (Interface Metaphor)

Ẩn dụ giao diện là việc sử dụng các khái niệm quen thuộc trong thế giới thực để mô phỏng hoặc minh họa cho các chức năng của hệ thống thông tin. Ví dụ: biểu tượng "giỏ hàng" trong các ứng dụng thương mại điện tử được dùng để thể hiện quá trình lựa chọn và lưu giữ sản phẩm trước khi thanh toán. Việc áp dụng các ẩn dụ này giúp người dùng hiểu nhanh hơn về chức năng của hệ thống và dễ dàng dự đoán hành vi của giao diện..

3.3.2 Đối tượng và Hành động Giao diện (Interface Objects and Actions)

Trong thiết kế giao diện, đối tượng (Objects) là những thành phần hoặc thực thể cơ bản mà người dùng có thể quan sát và tương tác trong hệ thống. Các đối tượng này có thể là:

- Biểu mẫu nhập liệu (Input form), ví dụ: form đăng ký tài khoản hoặc form đặt hàng.
- Bảng dữ liệu (Data table) hiển thị danh sách sản phẩm, khách hàng hoặc hóa đơn.
- Nút lệnh (Button), menu, biểu tượng (icon) hoặc hộp thoại (dialog box).

Hành động (Actions) là các thao tác hoặc lệnh mà người dùng có thể thực hiện đối với các đối tượng nói trên. Các hành động thường được thể hiện qua các nút, menu hoặc cử chỉ (trên thiết bị cảm ứng), chẳng hạn:

- "Thêm mới" (Add) để tạo một bản ghi mới.
- "Chỉnh sửa" (Edit) để thay đổi thông tin đã có.
- "Xóa" (Delete) để loại bỏ một mục khỏi danh sách.
- "Tìm kiếm" (Search) để tra cứu dữ liệu.
- "Mua hàng" (Buy) hoặc "Thanh toán" (Checkout) trong ứng dụng thương mại điện tử.

Việc lựa chọn và đặt tên cho các hành động cần tuân theo ngôn ngữ quen thuộc của người dùng, thay vì dùng thuật ngữ kỹ thuật. Ví dụ, trong hệ thống bán hàng, nên sử dụng nút "Mua hàng" thay vì "Purchase Item", hoặc "Thêm vào giỏ" thay vì "Add to cart" nếu người dùng chủ yếu là người Việt.

Bên cạnh đó, cần đảm bảo tính nhất quán trong việc đặt tên và vị trí của các hành động trên toàn hệ thống. Khi cùng một hành động xuất hiện ở nhiều nơi, nó nên có tên gọi, biểu tượng và vị trí tương tự nhau, giúp người dùng dễ ghi nhớ và thao tác nhanh hơn.

3.3.3 Biểu tượng (Icon)

Biểu tượng là những hình ảnh nhỏ đại diện cho đối tượng hoặc hành động và thường xuất hiện trên các nút lệnh, biểu mẫu và báo cáo để làm nổi bật thông tin quan trọng. Cách tiếp cận tốt nhất là áp dụng các biểu tượng đã được phát triển bởi những người khác (ví dụ: bút chì cho "chỉnh sửa"; thùng rác cho "xóa") vì người dùng đã quen thuộc với chúng.

3.3.4 Mẫu Giao diện (Interface Templates)

Mẫu giao diện xác định giao diện chung (general appearance) của tất cả các thành phần giao diện trong hệ thống (bao gồm màn hình, biểu mẫu và báo cáo), bao gồm bố cục cơ bản (vị trí khu vực điều hướng, trạng thái, v.v.), bảng màu (color scheme), và kiểu/kích thước phông chữ.

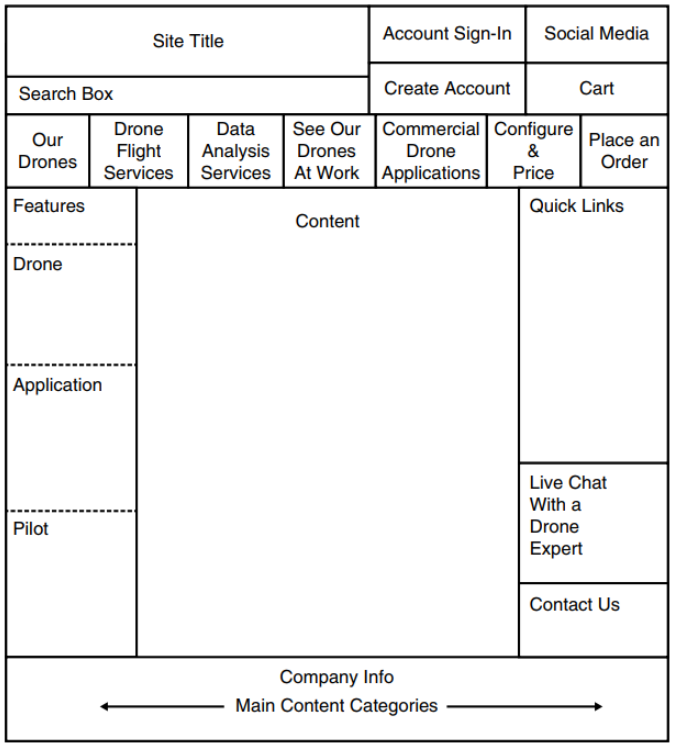
3.4 Tạo mẫu nguyên mẫu thiết kế UI

Nguyên mẫu thiết kế giao diện (interface design prototype) là một bản mô phỏng hoặc mô phỏng (mock-up or simulation) của màn hình máy tính, biểu mẫu hoặc báo cáo.

3.4.1 Sơ đồ Wireframe (Wireframe Diagram)

Được sử dụng để truyền tải nội dung và hành vi cơ bản của màn hình (thường là website)

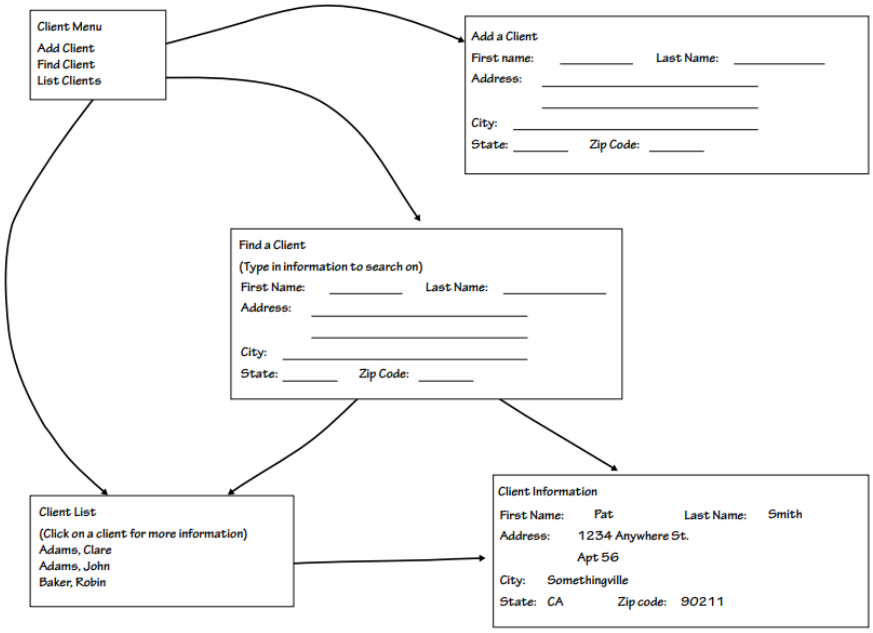
Mục tiêu là tập trung vào chức năng của màn hình, không phải giao diện trực quan của nó.



Hình 7 Ví dụ về wireframe

3.4.2 Bảng phân cảnh (Storyboard)

Là một nguyên mẫu dựa trên giấy đơn giản nhất. Nó hiển thị các bản vẽ phác thảo tay về giao diện màn hình sẽ trông như thế nào và cách chúng chuyển đổi từ màn hình này sang màn hình khác.



Hình 8 Ví dụ về storyboard

3.4.3 Sơ đồ Wireflow (Wireflow Diagram)

Là sự kết hợp giữa bảng phân cảnh và sơ đồ wireframe, giúp hình dung cách người dùng tương tác với một chuỗi màn hình để hoàn thành một tác vụ tổng thể.

Ngày nay, với các công cụ hiện tại như Figma, BA có thể dễ dàng tạo ra bản mock-up cho hệ thống cần xây dựng.

3.5 Đánh giá giao diện

3.5.1 Mục tiêu

Mục tiêu của hoạt động này là để xác định cách cải thiện thiết kế giao diện.

Mục tiêu và nguyên tắc chung của đánh giá giao diện:

- Tính chủ quan: Thiết kế giao diện là một quá trình chủ quan; không có công thức nào đảm bảo tạo ra một giao diện người dùng tuyệt vời.
- Thiết kế dựa trên Ưu tiên Người dùng: Hầu hết các nhà thiết kế thường thiết kế giao diện dựa trên sở thích cá nhân, điều này có thể không phù hợp với sở thích của người dùng.

- Nguyên tắc Quan trọng nhất: Thông điệp chính là cần có càng nhiều người càng tốt đánh giá giao diện—càng nhiều người dùng càng tốt. Hầu hết các chuyên gia khuyên nên có ít nhất 10 người dùng tiềm năng tham gia vào quá trình đánh giá.
- Thời điểm Lý tưởng: Lý tưởng nhất, việc đánh giá giao diện nên được thực hiện trong khi hệ thống đang được thiết kế—trước khi được xây dựng. Điều này giúp xác định và khắc phục các vấn đề thiết kế lớn trước khi tốn thời gian và chi phí lập trình.

3.5.2 Các phương pháp đánh giá

Có 4 phương pháp chính để đánh giá giao diện:

a) Đánh giá theo Heuristic

Phương pháp này kiểm tra giao diện bằng cách so sánh nó với một bộ các nguyên tắc (heuristics) đã định sẵn cho thiết kế giao diện. Nhóm dự án phát triển một danh sách kiểm tra các nguyên tắc thiết kế giao diện (ví dụ: các nguyên tắc về điều hướng, nhập liệu và xuất liệu). Ít nhất ba thành viên của nhóm dự án làm việc độc lập với bản prototype, kiểm tra từng giao diện để đảm bảo nó đáp ứng từng nguyên tắc trong danh sách. Sau đó, các thành viên gặp nhau để thảo luận về đánh giá của họ và xác định những cải tiến cụ thể.

Phương pháp này được coi là loại đánh giá yếu nhất vì nó không có sự tham gia của người dùng.

Tham khảo 10 nguyên tắc khả dụng cho thiết kế giao diện của Jakob Nielsen:

- *Hiển thị trạng thái hệ thống (Visibility of system status).* Giao diện nên luôn thông báo cho người dùng biết điều gì đang xảy ra, thông qua phản hồi kịp thời.
- *Phù hợp giữa hệ thống và thế giới thực (Match between system and the real world).* Sử dụng ngôn ngữ, khái niệm và quy ước quen thuộc với người dùng, không phải thuật ngữ kỹ thuật.
- *Kiểm soát và tự do của người dùng (User control and freedom).* Người dùng cần có lối thoát dễ dàng khi thực hiện nhầm thao tác.
- *Tính nhất quán và tiêu chuẩn (Consistency and standards).* Giao diện nên tuân theo các quy ước thiết kế phổ biến và nhất quán trong toàn hệ thống.
- *Phòng ngừa lỗi (Error prevention).* Thiết kế nên giúp người dùng không mắc lỗi ngay từ đầu, hơn là chỉ thông báo khi lỗi xảy ra.
- *Công nhận thay vì ghi nhớ (Recognition rather than recall).* Giảm gánh nặng trí nhớ cho người dùng bằng cách hiển thị các tùy chọn thay vì buộc họ phải nhớ.
- *Tính linh hoạt và hiệu quả sử dụng (Flexibility and efficiency of use).* Hệ thống nên hỗ trợ cả người mới và người dùng thành thạo.

- *Thẩm mỹ và tối giản (Aesthetic and minimalist design).* Chỉ hiển thị thông tin liên quan và cần thiết, tránh làm người dùng bị quá tải thông tin.
- *Giúp người dùng nhận ra, chẩn đoán và phục hồi lỗi (Help users recognize, diagnose, and recover from errors)*. hông báo lỗi phải rõ ràng, chỉ ra vấn đề và hướng dẫn cách khắc phục.
- *Trợ giúp và tài liệu (Help and documentation).* Tốt nhất là giao diện tự giải thích, nhưng vẫn cần cung cấp tài liệu hoặc hướng dẫn khi cần.

b) Đánh giá Walk-Through

Đây là một cuộc họp được tổ chức với những người dùng cuối cùng sẽ vận hành hệ thống. Nhóm dự án trình bày prototype (có thể là storyboard, hoặc trình diễn HTML hoặc ngôn ngữ prototype) và giải thích cách sử dụng giao diện. Người dùng đề xuất những cải tiến đối với từng giao diện được trình bày.

c) Đánh giá Tương tác

Người dùng tự mình làm việc với HTML hoặc ngôn ngữ prototype trong các phiên làm việc một đối một với các thành viên của nhóm dự án (không thể sử dụng storyboard). Ví dụ: sử dụng công cụ Figma. Khi người dùng tương tác với prototype (thường bằng cách thực hiện các use scenario hoặc tự do điều hướng), họ sẽ cho nhóm dự án biết những gì họ thích và không thích, cũng như những thông tin hoặc chức năng bổ sung cần thiết. Các thành viên nhóm ghi lại những lúc người dùng tỏ ra không chắc chắn về những gì cần làm, mắc lỗi hoặc hiểu sai ý nghĩa của một thành phần giao diện. Nếu mô hình không chắc chắn hoặc lỗi lặp lại trong nhiều phiên đánh giá, đó là dấu hiệu rõ ràng cho thấy những phần đó của giao diện cần được cải thiện.

d) Kiểm thử Khả năng Sử dụng Chính thức (Formal Usability Testing)

Đây là một quy trình rất chính thức—gần như mang tính khoa học—thường được thực hiện với các sản phẩm phần mềm thương mại hoặc các sản phẩm được phát triển bởi các tổ chức lớn. Chỉ có thể được sử dụng với ngôn ngữ prototype (hoặc các hệ thống đã được xây dựng hoàn chỉnh). Người dùng được giao một bộ tác vụ cụ thể để hoàn thành (thường là các use scenario). Các biện pháp đo lường hiệu suất được sử dụng bao gồm thời gian hoàn thành tác vụ, tỷ lệ lỗi và mức độ hài lòng của người dùng. Kiểm thử này khá tốn kém.

4 Ví dụ minh họa: Từ đặc tả Use Case đến thiết kế GUI

Phần này minh họa toàn bộ quy trình thiết kế UI vừa trình bày ở mục trên bằng một ví dụ cụ thể: use case "Đặt vé và chọn ghế" của hệ thống ThanhDat Ticket. Ví dụ đi từ sơ đồ use case và đặc tả use case, đầu ra của giai đoạn phân tích, qua bước xác định đối tượng và

hành động giao diện, sơ đồ cấu trúc giao diện, đến một wireframe cụ thể cho màn hình chính của use case này.

4.1 Bước 1: Xuất phát từ sơ đồ Use Case và đặc tả Use Case

Sơ đồ use case dưới đây trình bày use case "Đặt vé và chọn ghế" cùng actor Khách hàng và quan hệ «include»/«extend» với hai use case liên quan. Đây là một sơ đồ dạng nút và cạnh nên được trình bày dưới dạng mã Mermaid, có thể dán trực tiếp vào drawio (mục Extras, Edit Diagram, chọn định dạng Mermaid) để tiếp tục chỉnh sửa hình dạng và màu sắc:

```mermaid
flowchart LR
    KH(["Khách hàng"])
    UC1(["Tìm chuyến xe"])
    UC2(["Đặt vé và chọn ghế"])
    UC3(["Thanh toán vé"])
    UC4(["Áp dụng ưu đãi thành viên"])
    KH --- UC2
    UC2 -.->|"«include»"| UC1
    UC2 -.->|"«include»"| UC3
    UC4 -.->|"«extend»"| UC2
```

(Sau khi dán vào drawio, có thể thay các khối hình con nhộng bằng ký hiệu use case hình elip chuẩn UML và thêm hình que cho actor Khách hàng.)

Trường	**Nội dung**
Tên use case	Đặt vé và chọn ghế
Actor chính	Khách hàng
Mô tả ngắn	Khách hàng chọn ghế trống trên một chuyến xe đã tìm được và tạo một đơn đặt vé mới.
Điều kiện trước	Khách hàng đã thực hiện use case "Tìm chuyến xe" và chọn được một chuyến phù hợp.
Điều kiện sau	Một đơn đặt vé được tạo với trạng thái "Chờ thanh toán"; các ghế đã chọn chuyển trạng thái "Đang giữ".
Luồng chính	1. Hệ thống hiển thị sơ đồ ghế còn trống của chuyến xe.

	2. Khách hàng chọn một hoặc nhiều ghế trống. 3. Hệ thống giữ tạm các ghế đã chọn trong 10 phút. 4. Khách hàng xác nhận thông tin ghế đã chọn. 5. Hệ thống tạo đơn đặt vé và chuyển sang use case "Thanh toán vé".
Luồng ngoại lệ	3a. Ghế vừa được người khác đặt trước khi xác nhận: hệ thống thông báo và yêu cầu chọn lại. 4a. Hết thời gian giữ ghế: hệ thống tự động hủy giữ chỗ, khách phải chọn lại ghế.

4.2 Bước 2: Xác định đối tượng và hành động giao diện

Bảng dưới đây là bước cầu nối quan trọng: mỗi bước trong luồng chính của đặc tả use case được ánh xạ sang một đối tượng giao diện, một hành động và một điều khiển GUI cụ thể, theo đúng khái niệm "Đối tượng và Hành động Giao diện" đã trình bày ở mục trên.

Bước trong luồng chính	**Đối tượng giao diện**	**Hành động**	**Điều khiển GUI tương ứng**
1. Hiển thị sơ đồ ghế	Sơ đồ ghế (seat map)	Xem	Lưới ô ghế; mỗi ô là một nút bấm thể hiện trạng thái bằng màu
2. Chọn ghế trống	Ghế	Chọn	Nút bấm ghế, đổi màu khi được chọn
3. Giữ ghế tạm thời	Đơn đặt vé (nháp)	Giữ chỗ (tự động)	Đồng hồ đếm ngược hiển thị thời gian còn giữ ghế

4. Xác nhận thông tin	Thông tin ghế đã chọn	Xác nhận	Bảng tóm tắt liệt kê ghế, giá vé và nút "Tiếp tục"
5. Tạo đơn đặt vé	Đơn đặt vé	Tạo, chuyển tiếp	Nút "Tiếp tục thanh toán", điều hướng sang màn hình thanh toán

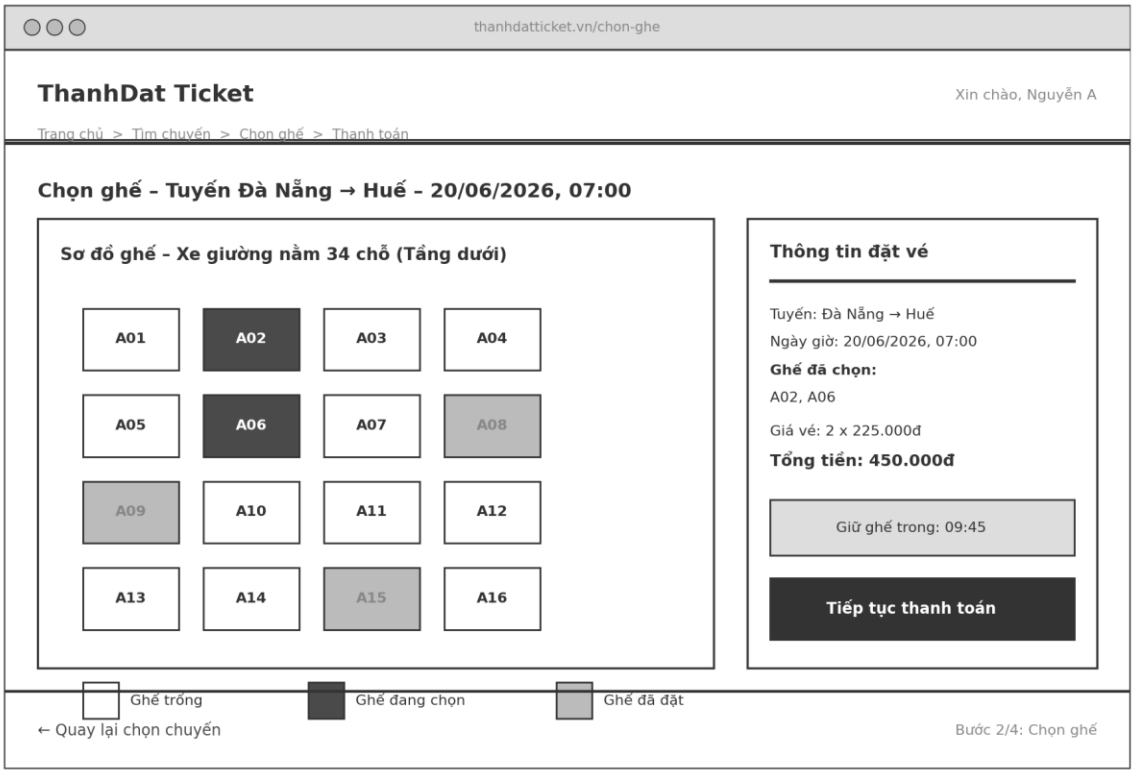
4.3 Bước 3: Sơ đồ cấu trúc giao diện (ISD) cho luồng use case

Sơ đồ cấu trúc giao diện dưới đây thể hiện luồng màn hình của toàn bộ use case, từ màn hình tìm chuyến đến màn hình vé điện tử. Đây cũng là một sơ đồ dạng nút và cạnh nên được trình bày dưới dạng Mermaid để tiện chỉnh sửa trong drawio:

```mermaid
flowchart TD
    S1["Màn hình: Tìm chuyến xe"]
    S2["Màn hình: Chọn ghế"]
    S3["Màn hình: Xác nhận thông tin"]
    S4["Màn hình: Thanh toán"]
    S5["Màn hình: Vé điện tử"]
    S1 -->|"Chọn chuyến"| S2
    S2 -->|"Xác nhận ghế"| S3
    S3 -->|"Tiếp tục thanh toán"| S4
    S3 -->|"Quay lại"| S2
    S4 -->|"Thanh toán thành công"| S5
    S4 -->|"Hủy"| S2
```

4.4 Bước 4: Wireframe minh họa cho màn hình chính

Trong bốn màn hình ở sơ đồ ISD trên, "Màn hình: Chọn ghế" là màn hình trung tâm của use case này. Hình dưới đây minh họa wireframe của màn hình đó, được xây dựng trực tiếp từ bảng đối tượng và hành động giao diện ở Bước 2.



Hình 9. Wireframe màn hình "Chọn ghế" của use case "Đặt vé và chọn ghế": lưới ô ghế bên trái thể hiện ba trạng thái (trống, đang chọn, đã đặt); bảng tóm tắt bên phải hiển thị thông tin chuyến, ghế đã chọn, tổng tiền, thời gian giữ ghế và nút điều hướng sang bước thanh toán.

Đối chiếu với bảng ở Bước 2 có thể thấy: lưới ô ghế hiện thực hóa đối tượng "Sơ đồ ghế" và hành động "Chọn"; ô màu tối thể hiện hành động chọn ghế đang diễn ra; dòng "Giữ ghế trong: 09:45" hiện thực hóa đối tượng "Đơn đặt vé (nháp)" và hành động "Giữ chỗ"; bảng tóm tắt bên phải hiện thực hóa đối tượng "Thông tin ghế đã chọn"; và nút "Tiếp tục thanh toán" hiện thực hóa hành động "Tạo, chuyển tiếp" sang màn hình kế tiếp trong sơ đồ ISD.

5 Thiết kế điều hướng (navigation)

Thành phần điều hướng của giao diện cho phép người dùng di chuyển khắp hệ thống, thực hiện các hành động và nhận các thông báo phản hồi khi cần. Mục tiêu của hệ thống điều hướng là làm cho hệ thống trở nên đơn giản nhất khi sử dụng. Một thành phần điều hướng tốt là thành phần mà người dùng hầu như không bao giờ thực sự nhận thấy. Nó hoạt động đúng như người dùng mong đợi, do đó người dùng ít phải bận tâm về nó.

5.1 Nguyên tắc

Một trong những thách thức lớn nhất khi sử dụng hệ thống máy tính là học cách thao tác với các điều khiển điều hướng để làm cho hệ thống hoạt động theo ý muốn. BA thường phải giả định rằng người dùng chưa đọc hướng dẫn sử dụng, chưa tham gia đào tạo, và

không có sự trợ giúp bên ngoài sẵn có. Do vậy, các điều khiển nên rõ ràng và dễ hiểu và được đặt ở một vị trí trực quan trên màn hình. Lý tưởng nhất là các điều khiển nên dự đoán được những gì người dùng sẽ làm và đơn giản hóa nỗ lực của họ. Ví dụ, sau khi đăng nhập vào trang web của ngân hàng, điều đầu tiên người dùng muốn thấy là danh sách tài khoản của họ, do vậy cần hiển thị danh sách tài khoản.

Các nguyên tắc cần tuân thủ:

- *Ngăn ngừa lỗi (Prevent Mistakes)*. Nguyên tắc đầu tiên của việc thiết kế các điều khiển điều hướng là ngăn người dùng mắc lỗi. Các lỗi mắc phải trong quá trình sử dụng hệ thống gây tốn thời gian và tạo ra sự thất vọng, và có thể khiến người dùng từ bỏ hệ thống.
    - Giảm lỗi bằng cách dán nhãn các lệnh và hành động một cách thích hợp và bằng cách hạn chế các lựa chọn. Quá nhiều lựa chọn có thể gây nhầm lẫn, đặc biệt khi chúng tương tự nhau và khó mô tả trong không gian ngắn có sẵn trên màn hình.
    - Không bao giờ hiển thị lệnh không thể sử dụng. Ví dụ, nhiều ứng dụng Windows làm mờ (gray-out) các lệnh không thể sử dụng; chúng được hiển thị bằng phông chữ màu rất nhạt, nhưng không thể chọn được
    - Khi người dùng sắp thực hiện một chức năng quan trọng khó hoặc không thể hoàn tác (ví dụ: xóa tệp), điều quan trọng là phải xác nhận hành động đó với người dùng.
- *Đơn giản hóa Khắc phục lỗi* (Simplify Recovery from Mistakes). Dù người thiết kế hệ thống làm gì đi nữa, người dùng vẫn có thể mắc lỗi. Hệ thống nên làm cho việc sửa lỗi trở nên dễ dàng nhất có thể. Lý tưởng nhất là hệ thống sẽ có nút "Hoàn tác" (Undo) giúp dễ dàng bỏ qua các lỗi, mặc dù việc viết code cho các nút như vậy có thể phức tạp.
- *Sử dụng Thứ tự Ngữ pháp Nhất quán* (Use Consistent Grammar Order). Trong thiết kế giao diện người dùng, thứ tự ngữ pháp đề cập đến cách sắp xếp giữa hành động (Action) và đối tượng (Object) trong các lệnh hoặc thao tác mà người dùng thực hiện. Phần lớn các lệnh trong hệ thống yêu cầu người dùng chỉ định một đối tượng (chẳng hạn như tệp tin, bản ghi, hoặc đoạn văn bản) và xác định hành động sẽ được thực hiện trên đối tượng đó (ví dụ: sao chép, xóa, di chuyển). Có hai cách tiếp cận phổ biến:
    - Thứ tự Đối tượng – Hành động (Object–Action Order): Người dùng chọn đối tượng trước, sau đó chọn hành động cần thực hiện. Ví dụ: Trong Windows, người dùng thường chọn tệp (object) trước, rồi nhấn Delete (action) để xóa tệp. Cách này giúp người dùng dễ hiểu vì họ “thấy đối tượng trước, hành

động sau", phù hợp với cách suy nghĩ tự nhiên khi thao tác trên giao diện trực quan.

- Thứ tự Hành động – Đối tượng (Action–Object Order): Người dùng chọn hành động trước, sau đó chỉ định đối tượng cần áp dụng. Ví dụ: Trong một số trình soạn thảo văn bản, người dùng có thể chọn lệnh Tìm kiếm (Find) rồi nhập từ khóa cần tìm (object).

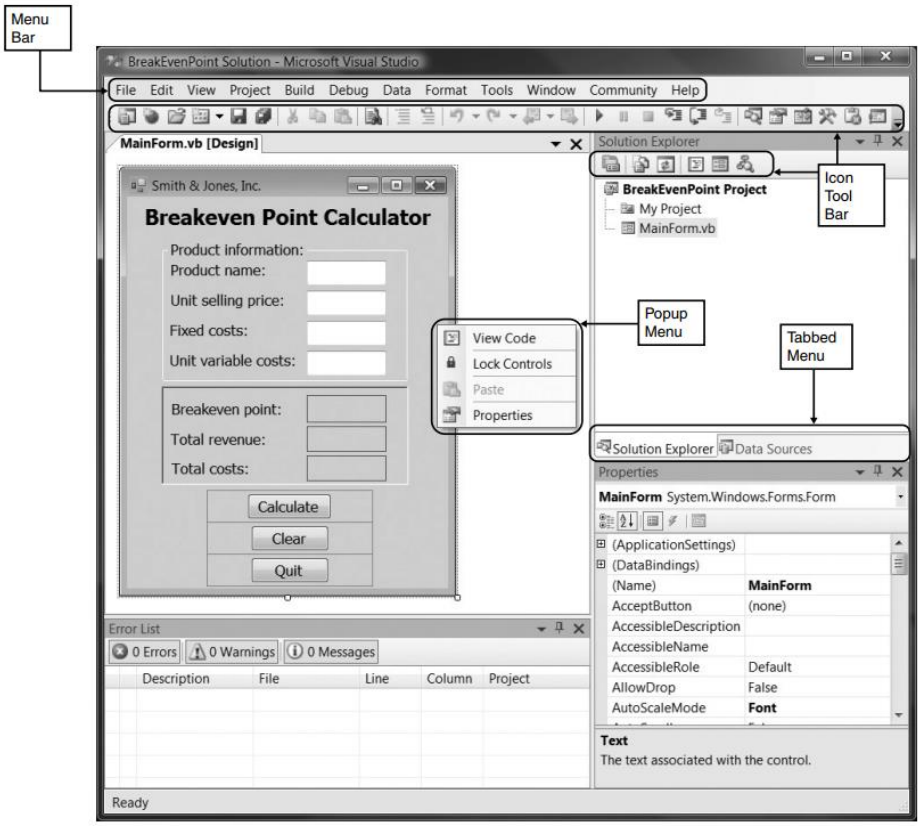
Dù sử dụng mô hình nào, tính nhất quán trong toàn bộ hệ thống là yếu tố bắt buộc. Thứ tự ngữ pháp phải được duy trì đồng nhất trên tất cả các màn hình, menu và hộp thoại,...Cả ở cấp độ phần tử dữ liệu (ví dụ: các nút, form) lẫn cấp độ cấu trúc tổng thể (ví dụ: menu hoặc thanh công cụ). Sự nhất quán này giúp người dùng nhanh chóng hình thành thói quen thao tác và giảm sai sót khi làm việc với hệ thống.

5.2 Menu Tips

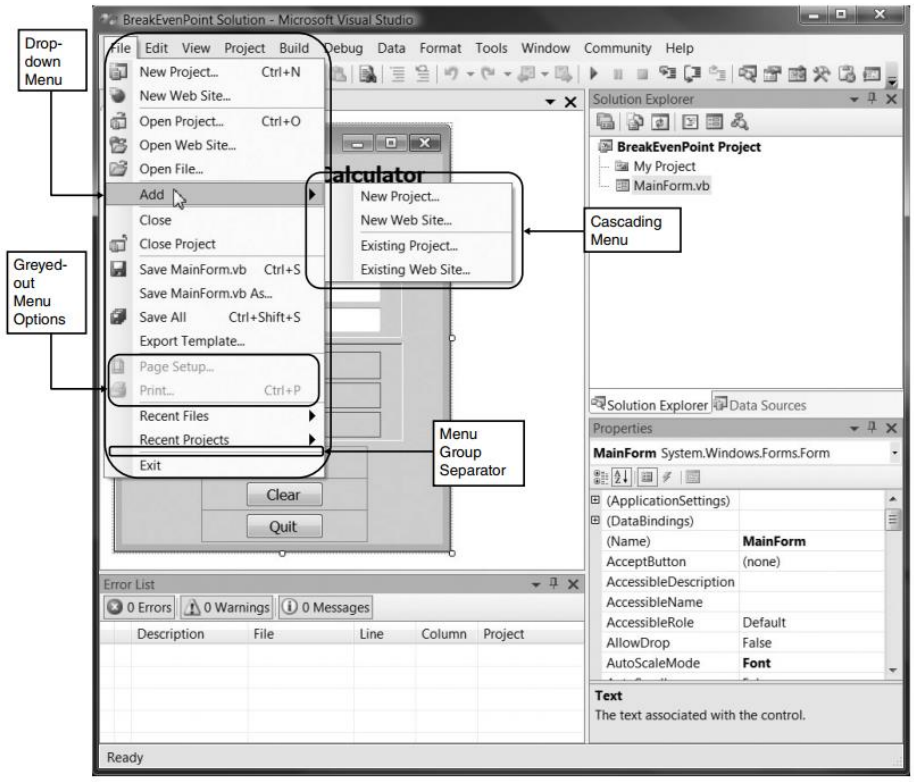
Menu là loại hệ thống điều hướng phổ biến nhất hiện nay. Menu trình bày cho người dùng một danh sách các lựa chọn, mỗi lựa chọn đều có thể được chọn. Menu dễ học vì người dùng thấy một tập hợp các lựa chọn có tổ chức.

- Nên thiết kế menu rộng và nông (nhiều mục trong mỗi menu, và mỗi mục chỉ dẫn đến một hoặc hai lớp menu) thay vì hẹp và sâu (ít mục, nhưng dẫn đến ba hoặc nhiều lớp menu). Menu rộng và nông trình bày cho người dùng nhiều thông tin nhất ban đầu, và chỉ yêu cầu ít lần nhấp chuột hoặc nhấn phím để thực hiện một hành động.
- Nên nhóm các mục menu lại với nhau và ngăn cách bằng một đường ngang nếu số lượng mục vượt quá mức lý tưởng (không quá tám mục trong một menu).
- Nên nhóm các loại mục tương tự lại với nhau trong một tùy chọn menu để người dùng có thể đoán một cách trực quan nội dung của tùy chọn đó. Hầu hết các nhà thiết kế khuyên nên nhóm các mục menu theo đối tượng giao diện (ví dụ: Khách hàng, Đơn đặt hàng, Hàng tồn kho) thay vì theo hành động tương tác (ví dụ: Thêm mới, Cập nhật, Định dạng).

Các loại menu phổ biến bao gồm thanh menu (menu bars), menu thả xuống (drop-down menus), menu bật lên (pop-up menus), menu tab (tab menus), thanh công cụ (tool bars), và bản đồ hình ảnh (image maps).



Hình 10 Ví dụ về menu - 1



Hình 11 Ví dụ về menu - 2

5.3 Message Tips

Thông báo là cách hệ thống phản hồi người dùng và thông báo cho họ về trạng thái của tương tác. Có nhiều loại thông báo khác nhau, chẳng hạn như thông báo lỗi (error messages), thông báo xác nhận (confirmation messages), thông báo thừa nhận (acknowledgment messages), thông báo trì hoãn (delay messages), và thông báo trợ giúp (help messages).

Nguyên tắc thiết kế thông báo:

- Rõ ràng, súc tích và đầy đủ
- Phải đúng ngữ pháp và không có biệt ngữ hoặc từ viết tắt (trừ khi đó là biệt ngữ hoặc từ viết tắt mà người dùng quen thuộc)
- Tránh dùng thể phủ định, vì chúng có thể gây nhầm lẫn
- Tránh dùng khiếu hài hước, vì nó sẽ nhanh chóng nhàm chán sau khi thông báo xuất hiện nhiều lần
- Thông báo nên yêu cầu người dùng xác nhận (trừ các tin nhắn thông báo về sự trì hoãn trong xử lý)
- Nên cung cấp số thông báo để hỗ trợ bộ phận trợ giúp xác định vấn đề

Các thông báo lỗi và trợ giúp:

- Thông báo lỗi phải luôn giải thích vấn đề bằng thuật ngữ lịch sự, ngắn gọn (ví dụ: người dùng đã làm sai điều gì) và giải thích hành động khắc phục một cách rõ ràng và minh bạch nhất có thể.
- Trong trường hợp lỗi phức tạp, thông báo lỗi nên hiển thị thông tin người dùng đã nhập, đề xuất các nguyên nhân có thể xảy ra cho lỗi và đề xuất các phản hồi có thể của người dùng.
- Thông báo trợ giúp (Help messages) cung cấp thông tin bổ sung về hệ thống và các thành phần của nó. Trợ giúp nhạy cảm theo ngữ cảnh (Context-sensitive help) cung cấp thông tin phụ thuộc vào hành động người dùng đang thực hiện khi yêu cầu trợ giúp.

6 Thiết kế đầu vào (input)

Thiết bị đầu vào tạo điều kiện cho việc nhập dữ liệu vào hệ thống máy tính, bao gồm cả dữ liệu có cấu trúc cao (ví dụ: thông tin đơn hàng) hoặc thông tin phi cấu trúc (ví dụ: bình luận). Thiết kế đầu vào bao gồm việc thiết kế các màn hình được sử dụng để nhập thông tin, cũng như bất kỳ biểu mẫu (form) nào mà người dùng viết hoặc gõ thông tin (ví dụ: bảng chấm công).

6.1 Nguyên tắc thiết kế đầu vào

Mục tiêu cốt lõi của thiết kế đầu vào là thu thập thông tin chính xác cho hệ thống một cách đơn giản và dễ dàng. Dưới đây là các nguyên tắc cơ bản:

6.1.1 Lựa chọn loại xử lý phù hợp

Có hai phương pháp chính để nhập dữ liệu vào hệ thống:

- Xử lý Trực tuyến (Online Processing): Còn gọi là xử lý giao dịch trực tuyến (Online Transaction Processing – OLTP). Ở phương pháp này, mỗi giao dịch được nhập và xử lý ngay lập tức tại thời điểm nó diễn ra. Điều này giúp hệ thống cập nhật dữ liệu theo thời gian thực (real-time), đảm bảo thông tin luôn mới nhất. Phương pháp này thường được sử dụng trong các hoạt động mà tốc độ và độ chính xác của thông tin tức thời rất quan trọng, chẳng hạn như:
    - Giao dịch ngân hàng trực tuyến,
    - Hệ thống bán hàng (POS),
    - Đặt vé máy bay hoặc đặt phòng khách sạn.
- Xử lý Theo lô (Batch Processing): Ở phương pháp này, các giao dịch không được xử lý ngay lập tức mà được thu thập và lưu trữ trong một khoảng thời gian nhất định, sau đó xử lý đồng loạt trong một lần. Cách làm này phù hợp với các quy trình không yêu cầu thông tin thời gian thực, giúp tiết kiệm tài nguyên hệ thống và nâng cao hiệu suất. Ví dụ:
    - Tính lương nhân viên vào cuối tháng,
    - Tổng hợp dữ liệu bán hàng hằng ngày,
    - Lưu trữ và xử lý dữ liệu thống kê định kỳ.

6.1.2 Thu thập Dữ liệu tại Nguồn (Capture Data at the Source)

Nguyên tắc quan trọng nhất của thiết kế đầu vào là thu thập dữ liệu ở định dạng điện tử tại nguồn gốc ban đầu hoặc càng gần nguồn càng tốt. Hầu hết các hệ thống xử lý giao dịch ngày nay sử dụng tự động hóa dữ liệu nguồn bằng cách sử dụng các thiết bị phần cứng đặc biệt để thu thập dữ liệu. Các ví dụ về công nghệ tự động hóa dữ liệu nguồn bao gồm máy đọc mã vạch (bar code readers), nhận dạng ký tự quang học (optical character recognition), máy đọc dải từ tính (magnetic stripe readers), thẻ thông minh (smart cards), và thẻ nhận dạng tần số vô tuyến (RFID) tag. Việc thu thập dữ liệu tại nguồn giảm thời gian và chi phí nhập liệu, đồng thời giảm lỗi vì ít có khả năng thu thập dữ liệu sai sót.

6.1.3 Giảm thiểu Thao tác Gõ phím (Minimize Keystrokes)

Một nguyên tắc quan trọng khác là giảm thiểu thao tác gõ phím. Thao tác gõ phím tốn thời gian và thường xuyên không chính xác.

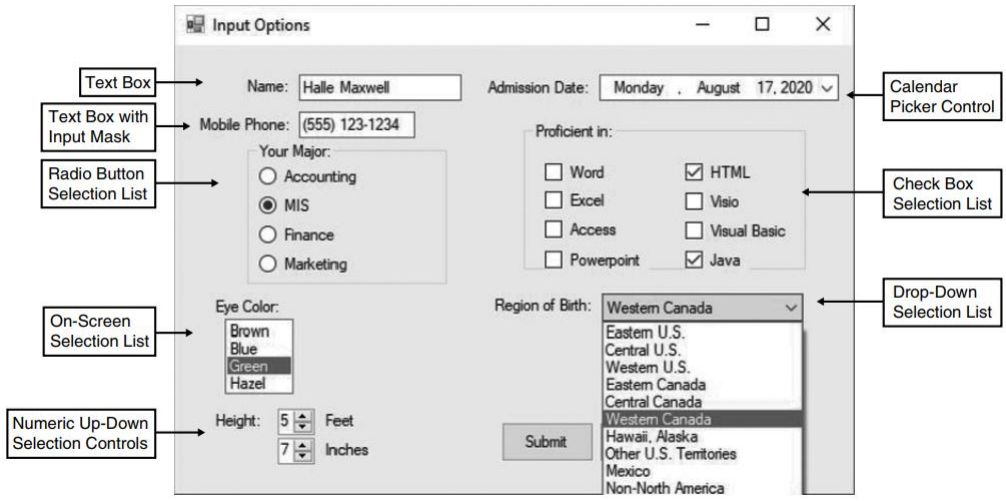
- Hệ thống không bao giờ yêu cầu thông tin có thể lấy được bằng cách khác (ví dụ: truy xuất từ cơ sở dữ liệu hoặc thực hiện phép tính).
- Hệ thống không nên yêu cầu người dùng gõ thông tin có thể được chọn từ một danh sách, vì việc chọn giúp giảm lỗi và tăng tốc độ nhập liệu.
- Các giá trị mặc định (default values) cho trường nên được sử dụng khi giá trị có khả năng xảy ra nhất có thể được dự đoán, để người dùng có thể chấp nhận giá trị đó mà không cần gõ lại.

6.2 Các gợi ý về giá trị đầu vào

Khi tạo form, việc nhập dữ liệu mới bằng bàn phím nên được giữ ở mức tối thiểu, vì lỗi thường xảy ra khi gõ. Nên sử dụng các điều khiển lựa chọn đầu vào bất cứ khi nào có thể, đặc biệt đối với các ứng dụng màn hình nhỏ, có cảm ứng.

6.2.1 Nhập Văn bản (Entering Text)

Khi cần nhập văn bản từ bàn phím, hộp văn bản (text box) được sử dụng. Trong ứng dụng không cảm ứng, nhãn trường (field label) nên đặt bên trái hộp văn bản; trong ứng dụng cảm ứng, nên đặt nhãn dọc theo phía trên hộp văn bản. Nếu đầu vào là giá trị số có định dạng chuẩn (ví dụ: số điện thoại), nên áp dụng mặt nạ đầu vào (input mask) cho hộp văn bản.



Hình 12 Ví dụ về các loại input controls

6.2.2 Thực hiện Lựa chọn (Making Selections)

Nhiều tùy chọn có sẵn để người dùng chọn giá trị đầu vào mong muốn:

- Bộ chọn lịch (Calendar picker control) nên được sử dụng để chọn ngày thay vì nhập số.

- Điều khiển số tăng/giảm (Numeric up-down selectors) có thể được sử dụng để chọn các giá trị số. Chúng phù hợp nhất để nhập các giá trị có phạm vi hẹp, chẳng hạn như "Số lượng Đã đặt hàng".
- Danh sách lựa chọn (Selection lists) cho phép người dùng chọn một hoặc nhiều giá trị từ danh sách được xác định trước. Các mục nên được sắp xếp theo thứ tự có ý nghĩa, như thứ tự bảng chữ cái hoặc thứ tự mục được sử dụng thường xuyên nhất.

Các loại danh sách lựa chọn phổ biến bao gồm:

- Hộp kiểm (Check box selection list): Khi có thể chọn nhiều mục từ danh sách các mục. Hộp kiểm không loại trừ lẫn nhau (not mutually exclusive).
- Nút radio (Radio button selection list): Khi chỉ có thể chọn một mục.
- Danh sách lựa chọn thả xuống (Drop-down selection list): Được sử dụng khi không đủ chỗ để hiển thị tất cả các lựa chọn.
- Hộp kết hợp (Combo box selection list): Một loại danh sách thả xuống đặc biệt cho phép người dùng vừa gõ vừa cuộn danh sách. Đây là một lối tắt cho người dùng có kinh nghiệm khi danh sách dài

6.3 Xác thực đầu vào

Tất cả dữ liệu được nhập vào hệ thống phải được xác thực để đảm bảo độ chính xác. Xác thực đầu vào được sử dụng để ngăn thông tin không hợp lệ xâm nhập vào hệ thống. Có sáu loại kiểm tra xác thực:

Loại Kiểm tra	Mục đích
Kiểm tra tính đầy đủ (Completeness check)	Đảm bảo rằng tất cả dữ liệu bắt buộc đã được nhập. Nếu thiếu, form sẽ được trả lại.
Kiểm tra định dạng (Format check)	Đảm bảo rằng dữ liệu đúng loại (ví dụ: số) và đúng định dạng (ví dụ: tháng, ngày, năm).
Kiểm tra phạm vi (Range check)	Đảm bảo rằng giá trị nằm trong phạm vi cho phép.
Kiểm tra chữ số kiểm tra (Check digit check)	Được sử dụng để kiểm tra lỗi nhập liệu khi nhập số nhận dạng (identification numbers).
Kiểm tra tính nhất quán (Consistency check)	Đảm bảo rằng sự kết hợp của các trường là hợp lý (ví dụ: ngày giao hàng không thể trước ngày đặt hàng).
Kiểm tra cơ sở dữ liệu (Database check)	So sánh dữ liệu đầu vào với dữ liệu đã có trong cơ sở dữ liệu (ví dụ: kiểm tra xem mã khách hàng có tồn tại không).

Lý tưởng nhất, hệ thống nên thực hiện tất cả các kiểm tra thích hợp nếu có thể. Các kiểm tra cơ sở dữ liệu thường được thực hiện sau các kiểm tra khác vì sẽ "tốn kém" hơn (yêu cầu hệ thống làm nhiều việc hơn).

7 Thiết kế đầu ra (output)

Đầu ra là các báo cáo hoặc thông tin mà hệ thống tạo ra, có thể hiển thị trên màn hình, in ra giấy, hoặc ở các phương tiện khác như web. Đầu ra là phần dễ thấy nhất của bất kỳ hệ thống nào, vì mục đích chính của việc sử dụng một hệ thống thông tin là để truy cập thông tin mà nó tạo ra.

7.1 Nguyên tắc thiết kế đầu ra

Mục tiêu của cơ chế đầu ra là trình bày thông tin cho người dùng sao cho họ có thể hiểu thông tin đó một cách chính xác với ít nỗ lực nhất. Các nguyên tắc thiết kế cơ bản phản ánh cách các đầu ra được sử dụng và làm thế nào để người dùng dễ dàng hiểu chúng hơn.

7.1.1 Hiểu cách sử dụng báo cáo (Understand Report Usage)

Nguyên tắc đầu tiên trong việc thiết kế báo cáo là hiểu cách chúng được sử dụng.

- Mục đích sử dụng: Trong hầu hết các trường hợp, báo cáo được sử dụng để xác định các mục cụ thể hoặc được dùng làm tài liệu tham khảo để tìm kiếm thông tin.
- Thứ tự sắp xếp quan trọng: Thứ tự sắp xếp các mục trên báo cáo hoặc cách nhóm chúng theo danh mục là rất quan trọng.
- Thiết kế cho web: Các báo cáo trên web hoặc báo cáo điện tử nhằm mục đích đọc toàn bộ nên được trình bày trên một trang duy nhất thay vì bị chia thành nhiều trang với các liên kết riêng biệt. Số trang và ngày báo cáo được lập cũng quan trọng đối với các báo cáo tham chiếu.

7.1.2 Sử dụng thông tin chính xác (Provide the Right Information)

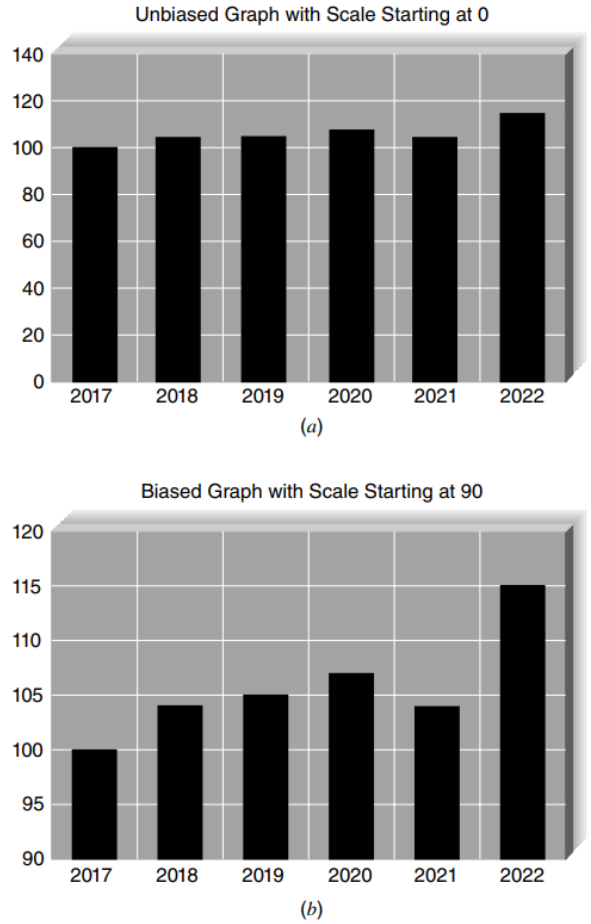
Các báo cáo phải cung cấp tất cả (và chỉ) thông tin cần thiết để hỗ trợ nhiệm vụ mà chúng được thiết kế cho. Điều này không có nghĩa là báo cáo nên cung cấp tất cả thông tin có sẵn về chủ đề đó, mà chỉ là những gì người dùng cần để thực hiện công việc của họ. Đôi khi, điều này dẫn đến việc tạo ra nhiều báo cáo khác nhau về cùng một chủ đề cho cùng một người dùng, vì chúng được sử dụng theo những cách khác nhau. Đây không phải là một thiết kế tồi.

7.1.3 Cung cấp dữ liệu dễ hiểu (Present Information Effectively)

Báo cáo nên trình bày dữ liệu bằng cách sử dụng các tiêu đề, nhãn (labels), và định dạng trực quan để người dùng dễ hiểu nhất có thể.

- Việc sử dụng các nguyên tắc thiết kế giao diện (như Bố cục, Nhận thức Nội dung, Tính Thẩm mỹ, Tính Nhất quán) là rất quan trọng khi thiết kế các đầu ra.
- Định dạng dữ liệu: Dữ liệu số nên được trình bày theo định dạng chuẩn, có dấu phẩy phân cách hàng nghìn và hiển thị hai chữ số thập phân, đồng thời các số nên được căn lề phải.

- Biểu đồ (Graphs): Biểu đồ nên được sử dụng bất cứ khi nào có thể để làm cho dữ liệu dễ hiểu hơn. Tuy nhiên, cần lưu ý rằng biểu đồ có thể bị thiên vị (biased) nếu thiết kế kém.



Hình 13 Ví dụ về biểu đồ thiên vị và không thiên vị

7.2 Các loại đầu ra

Có nhiều loại báo cáo khác nhau, mặc dù việc phân loại có thể khó khăn vì nhiều báo cáo có đặc điểm của nhiều loại cùng lúc.

Loại Đầu ra	Mục đích	Mô tả và Ví dụ
Báo cáo Chi tiết (Detail Report)	Cung cấp thông tin chi tiết về các giao dịch hoặc sự kiện.	Mỗi dòng trên báo cáo tương ứng với một bản ghi. Báo cáo này thường được đọc để tìm một bản ghi cụ thể hoặc để so sánh các bản ghi.
Báo cáo Tổng kết (Summary Report)	Tổng hợp thông tin.	Được sử dụng khi người dùng cần thông tin về tổng thể hệ thống, ví dụ: tổng doanh số, tổng chi phí. Báo cáo này thường được đọc để so sánh một số mục với nhau, do đó thứ tự sắp xếp là quan trọng.
Báo cáo Ngoại lệ (Exception Report)	Liệt kê các điều kiện hoặc giao dịch bất thường.	Được sử dụng khi người dùng quan tâm đến các sự kiện đặc biệt, ví dụ: các đơn đặt hàng quá hạn, các giao dịch lớn hơn một giới hạn định sẵn. Loại báo

		cáo này giúp giảm tải thông tin, cho phép người dùng tập trung vào các vấn đề quan trọng nhất.
Tài liệu Hoàn thành/Lưu thông (Turnaround Document)	Các đầu ra mà sau đó được **"quay vòng" (turn around)** và trở thành đầu vào.	Ví dụ: hầu hết các hóa đơn gửi cho khách hàng (như hóa đơn thẻ tín dụng) cung cấp thông tin về tổng số tiền nợ và chứa một biểu mẫu mà người tiêu dùng điền vào và gửi lại cùng với thanh toán.
Biểu đồ (Graphs)	Giúp trình bày dữ liệu một cách trực quan để người dùng dễ hiểu hơn.	Có thể là báo cáo độc lập hoặc là một phần của báo cáo khác.

7.3 Phương tiện

Ngày nay, nhiều tổ chức tạo ra báo cáo dưới dạng điện tử. Lợi ích của báo cáo điện tử: Báo cáo được "in" nhưng lưu trữ dưới định dạng điện tử trên máy chủ tệp (file servers) hoặc máy chủ Web, điều này giúp:

- Người dùng có thể dễ dàng truy cập.
- Thường có sẵn ở nhiều định dạng được thiết kế sẵn (vì chi phí lưu trữ là tối thiểu).
- Có thể được tạo theo yêu cầu (on demand).
- Cho phép người dùng tìm kiếm từ khóa dễ dàng hơn

Đối với các trường hợp đặc biệt: Mặc dù báo cáo điện tử phổ biến, báo cáo trên giấy vẫn được sử dụng khi:

- Cần phân phối đến nhiều người dùng (ví dụ: thư quảng cáo).
- Yêu cầu chữ ký hoặc xác nhận (ví dụ: tài liệu hoàn thành/lưu thông).
- Cần tuân thủ các quy định hoặc luật pháp (ví dụ: bản sao thuế)

8 Các công cụ phổ biến dùng trong thiết kế giao diện

Hiện nay, công cụ phổ biến được các công ty sử dụng trong thiết kế giao diện là Figma. Các bạn có thể tham khảo hướng dẫn sử dụng Figma tại link sau:

https://www.youtube.com/watch?v=f1VYYjEhX2Q&list=PLZ52njWO5oxKNf7ZxH1YjH4lU80QefcZH&index=2

9 Xu hướng và công nghệ mới trong thiết kế giao diện

9.1 Giới thiệu

Thiết kế giao diện người dùng (User Interface Design – UI) không ngừng thay đổi theo sự phát triển của công nghệ, thiết bị, và hành vi người dùng. Nếu trước đây UI chỉ đơn thuần là các biểu mẫu nhập liệu trong hệ thống quản lý, thì nay nó đã trở thành trải nghiệm số toàn diện (Digital Experience), trải dài từ web, di động, thiết bị đeo, đến hệ thống thông minh sử dụng trí tuệ nhân tạo.

Các xu hướng mới trong thiết kế giao diện hiện đại hướng tới ba mục tiêu:

- Cá nhân hóa trải nghiệm (Personalization).
- Tương tác tự nhiên và linh hoạt (Natural Interaction).
- Kết nối liền mạch giữa các nền tảng (Seamless Multi-platform Experience).

9.2 Thiết kế giao diện thích ứng (Responsive & Adaptive Design)

9.2.1 Khái niệm

Thiết kế thích ứng (Responsive Design) là kỹ thuật cho phép giao diện tự động điều chỉnh bố cục, kích thước và thành phần để phù hợp với mọi loại thiết bị (máy tính, máy tính bảng, điện thoại).

Thiết kế thích ứng (Adaptive Design) đi xa hơn – nó sử dụng bố cục được tối ưu hóa riêng cho từng loại thiết bị, thay vì chỉ co giãn linh hoạt.

9.2.2 Đặc điểm chính

- Lưới linh hoạt (Fluid Grid): các phần tử giao diện được định kích thước bằng tỷ lệ phần trăm, không cố định pixel.
- Hình ảnh co giãn (Flexible Images): tự điều chỉnh kích thước và độ phân giải.
- Điểm dừng (Breakpoints): giao diện thay đổi bố cục ở các kích thước màn hình nhất định.
- Kiểm thử đa nền tảng: đảm bảo giao diện hiển thị tốt trên tất cả thiết bị và trình duyệt.

9.2.3 Lợi ích

- Trải nghiệm người dùng nhất quán trên mọi thiết bị.
- Giảm chi phí phát triển, không cần tạo nhiều phiên bản giao diện.
- Cải thiện thứ hạng SEO (Google ưu tiên trang web thân thiện di động).

Ví dụ: Trang đăng ký học phần hiển thị ba cột trên máy tính, nhưng tự động chuyển thành một cột dọc dễ đọc trên điện thoại.

9.3 Thiết kế hướng trải nghiệm người dùng (UX-Centered Design)

Nếu UI (User Interface) tập trung vào giao diện, thì UX (User Experience) tập trung vào trải nghiệm tổng thể của người dùng khi tương tác với sản phẩm. Thiết kế hiện đại luôn kết hợp cả hai, hình thành UX/UI Design như một lĩnh vực chuyên biệt.

9.3.1 Các giai đoạn trong UX Design

- Nghiên cứu người dùng (User Research): hiểu hành vi, mục tiêu, nỗi đau (pain points).
- Thiết kế hành trình người dùng (User Journey Map): mô tả cảm xúc và thao tác của người dùng trong suốt quá trình sử dụng.
- Xây dựng nguyên mẫu và kiểm thử (Prototyping & Testing): kiểm tra giả thuyết thiết kế qua phản hồi thực tế.
- Đánh giá và cải tiến liên tục (Continuous Improvement): đo lường chỉ số UX (hài lòng, giữ chân người dùng).

9.3.2 Vai trò trong phát triển hệ thống thông tin

Thiết kế UX giúp hệ thống không chỉ đúng yêu cầu nghiệp vụ, mà còn được người dùng yêu thích và sử dụng hiệu quả – yếu tố quyết định thành công khi triển khai hệ thống thực tế trong doanh nghiệp.

9.4 Giao diện sử dụng trí tuệ nhân tạo (AI-driven Interfaces)

Giao diện điều khiển bởi AI là loại giao diện có khả năng học hỏi từ hành vi người dùng và tự động thích ứng, cung cấp thông tin hoặc hành động được cá nhân hóa.

Ứng dụng phổ biến:

- Gợi ý thông minh (Smart Recommendation): Ví dụ: hệ thống học tập trực tuyến đề xuất khóa học phù hợp dựa trên lịch sử học tập.
- Tự động hoàn thành và dự đoán (Predictive Input): Gợi ý văn bản hoặc dữ liệu khi người dùng nhập.
- Phân tích hành vi (Behavioral Analytics): Theo dõi hành vi sử dụng để điều chỉnh bố cục hoặc nội dung hiển thị.
- Trợ lý ảo (Virtual Assistant / Chatbot): Cung cấp hỗ trợ tức thì, hướng dẫn người dùng và xử lý yêu cầu tự động.

9.5 Giao diện hội thoại (Conversational Interface)

Là loại giao diện cho phép người dùng giao tiếp với hệ thống bằng ngôn ngữ tự nhiên, dưới dạng chat hoặc giọng nói. Hệ thống sử dụng công nghệ Xử lý ngôn ngữ tự nhiên (NLP) để hiểu và phản hồi.

Các dạng phổ biến

- Chatbot văn bản: hỗ trợ trực tuyến trong web/app.
- Voice Assistant: giao diện điều khiển bằng giọng nói (Siri, Alexa, Google Assistant).
- AI Tutor / AI Support Agent: hướng dẫn người dùng thực hiện tác vụ.

9.6 Thiết kế trải nghiệm đa nền tảng (Cross-platform & Omnichannel UX)

Người dùng ngày nay sử dụng nhiều thiết bị khác nhau (máy tính, điện thoại, tablet, smartwatch, thiết bị IoT). Do đó, giao diện cần được thiết kế để đảm bảo trải nghiệm liền mạch giữa các nền tảng.

Nguyên tắc thiết kế:

- Nhất quán về thương hiệu: màu sắc, biểu tượng, ngôn ngữ thiết kế.
- Chuyển đổi mượt mà: người dùng có thể bắt đầu tác vụ trên một thiết bị và hoàn tất trên thiết bị khác.
- Đồng bộ dữ liệu thời gian thực.

Ví dụ: Người dùng đặt vé máy bay trên website → tiếp tục thanh toán qua ứng dụng di động → nhận thông tin qua email và đồng hồ thông minh.

9.7 Xu hướng giao diện tương lai

Xu hướng	Mô tả	Tác động đến thiết kế
Giao diện 3D và thực tế tăng cường (AR/VR UI)	Môi trường tương tác ba chiều	Cần thiết kế không gian, cử chỉ, độ sâu
Giao diện không chạm (Touchless UI)	Nhận dạng chuyển động, ánh mắt, cảm xúc	Tăng tính tự nhiên và an toàn (hậu COVID-19)
Micro-interactions	Các hiệu ứng nhỏ (rung, chuyển động, màu sáng) khi thao tác	Tăng tính sinh động và phản hồi cảm xúc
Thiết kế tối giản (Minimalist Design)	Giảm chi tiết thừa, tập trung nội dung chính	Dễ đọc, dễ thao tác, hiệu suất cao
Dark Mode	Giao diện nền tối	Giảm mỏi mắt, tiết kiệm năng lượng

10 Tài liệu tham khảo

Alan Dennis, Barbara Haley Wixom, David Tegarden (2021). Systems *Analysis and Design: An Object-Oriented Approach with UML*. Wiley.